

Name: Aditi Laxman Bibave

Roll no: 08

Class: BE Computer

Assignment 2A — Binary Classification using Deep Neural Network

Dataset : IMDB Movie Reviews (built into Keras)

Task : Classify reviews as Positive (1) or Negative (0)

THEORY: Binary classification maps input → one of two classes.

The IMDB dataset contains 50,000 movie reviews, each pre-tokenised into integers representing word indices.

```
# Import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from tensorflow.keras.datasets import imdb
from tensorflow.keras import models
from tensorflow.keras import layers

# Load IMDB dataset (top 10,000 words)
(X_train, y_train), (X_test, y_test) = imdb.load_data(num_words=10000)

# Combine for EDA
data = np.concatenate((X_train, X_test), axis=0)
labels = np.concatenate((y_train, y_test), axis=0)

# -----
# EDA (Exploratory Analysis)
# -----

print("Categories:", np.unique(labels))
```

```
print("Number of unique words:", len(np.unique(np.hstack(data))))
```

```
# Count Plot (like PDF)
```

```
label_df = pd.DataFrame({'label': labels})
```

```
sns.countplot(x='label', data=label_df)
```

```
plt.title("Positive vs Negative Reviews")
```

```
plt.show()
```

```
# Review length analysis
```

```
review_lengths = [len(i) for i in data]
```

```
print("Average Review Length:", np.mean(review_lengths))
```

```
print("Standard Deviation:", np.std(review_lengths))
```

```
# Box Plot (extra + good for marks)
```

```
sns.boxplot(x=review_lengths)
```

```
plt.title("Review Length Distribution")
```

```
plt.show()
```

```
# -----
```

```
# Vectorization
```

```
# -----
```

```
def vectorize(sequences, dimension=10000):
```

```
    results = np.zeros((len(sequences), dimension))
```

```
    for i, sequence in enumerate(sequences):
```

```
        results[i, sequence] = 1
```

```
    return results
```

```
data = vectorize(data)
```

```
labels = np.asarray(labels).astype("float32")
```

```
# Split into train and test
```

```
X_train = data[10000:]
```

```
X_test = data[:10000]
```

```
y_train = labels[10000:]
```

```
y_test = labels[:10000]
```

```
# -----
```

```

# Build Model
# -----

model = models.Sequential()

# Input Layer
model.add(layers.Dense(50, activation='relu', input_shape=(10000,)))

# Hidden Layers
model.add(layers.Dropout(0.3))
model.add(layers.Dense(50, activation='relu'))
model.add(layers.Dropout(0.2))
model.add(layers.Dense(50, activation='relu'))

# Output Layer
model.add(layers.Dense(1, activation='sigmoid'))


# Compile model
model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])
# -----

# Train Model
# -----

history = model.fit(X_train, y_train,
                   epochs=2,
                   batch_size=500,
                   validation_data=(X_test, y_test))
# -----

# Evaluate Model
# -----

score = model.evaluate(X_test, y_test)
print("Test Loss:", score[0])
print("Test Accuracy:", score[1])
# -----

# Plot Accuracy
# -----

```

```

plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend(['Train', 'Test'])
plt.show()

# -----

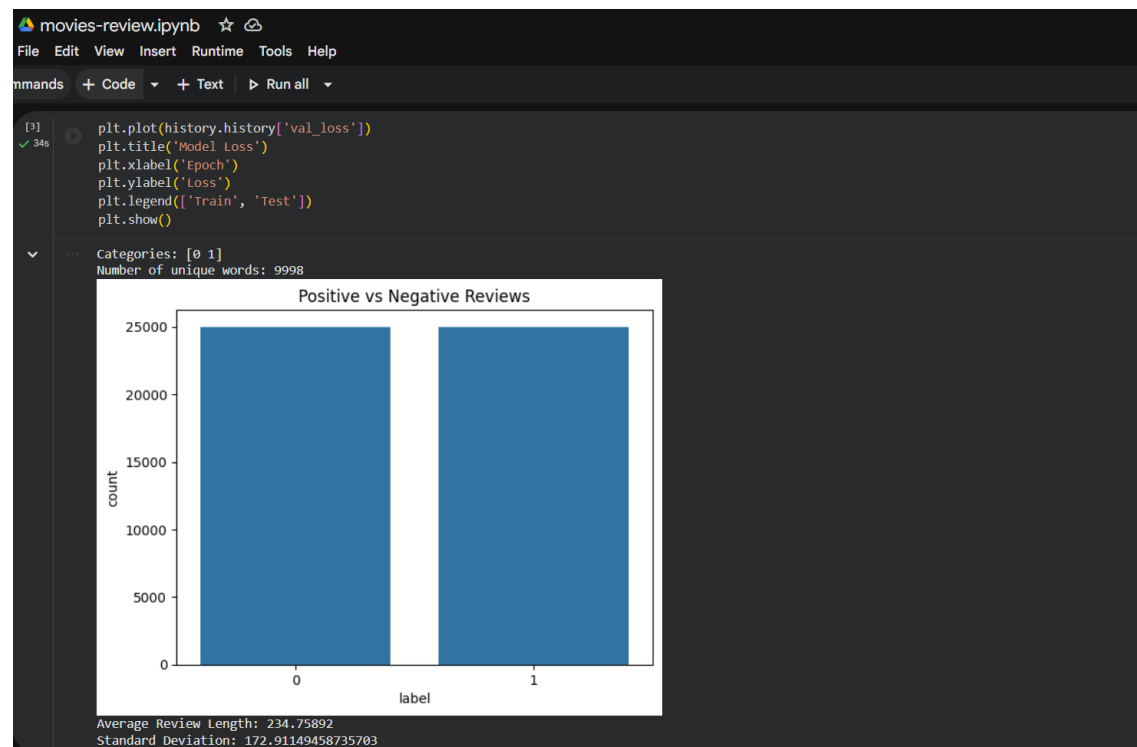
# Plot Loss

# -----

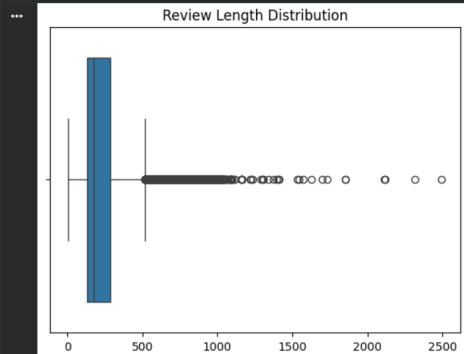
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend(['Train', 'Test'])
plt.show()

```

OUTPUT:



```
plt.plot(history.history['val_loss'])
plt.title('Model Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend(['Train', 'Test'])
plt.show()
```



/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an "input_shape"/"input_dim" argument to a layer. When using Sequential

super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Epoch 1/2

80/80 9s 90ms/step - accuracy: 0.8262 - loss: 0.3957 - val_accuracy: 0.8923 - val_loss: 0.2625

Epoch 2/2

80/80 5s 60ms/step - accuracy: 0.9196 - loss: 0.2129 - val_accuracy: 0.8954 - val_loss: 0.2644

80/80 9s 90ms/step - accuracy: 0.8262 - loss: 0.3957 - val_accuracy: 0.8923 - val_loss: 0.2625

Epoch 2/2

80/80 5s 60ms/step - accuracy: 0.9196 - loss: 0.2129 - val_accuracy: 0.8954 - val_loss: 0.2644

...

313/313 2s 6ms/step - accuracy: 0.8954 - loss: 0.2644

Test Loss: 0.2643871009349823

Test Accuracy: 0.8953999876976013

